

Техническое задание: Аппаратно-программная реализация игры «Сапёр»

1. Общее описание проекта

Вашей задачей является проектирование и реализация игровой системы «Сапёр» в среде Logisim на базе 8-разрядного микропроцессора CdM-8. Проект представляет собой совместную разработку вычислительного ядра, специализированных периферийных контроллеров ввода-вывода (Hardware) и управляющей программы на языке ассемблера CdM-8 (Software).

Игра разворачивается на квадратном поле размером 8*8 ячеек (всего 64 клетки). Система использует модель прерываний для обработки действий пользователя и отображает графику на светодиодных матричных дисплеях высокого разрешения.

2. Игровые правила и логика

2.1. Инициализация

На старте игры автоматически генерируется или загружается из памяти фиксированная карта. На поле размещается ровно 10 мин. Игроку выдается стартовый запас из 10 флажков. Курсор по умолчанию устанавливается в верхний левый угол (ячейка 0).

2.2. Управление

Перемещение курсора по полю осуществляется по осям X и Y. При достижении края поля курсор циклически перемещается на противоположную сторону.

2.3. Действие «Открыть ячейку»

Если в ячейке находится мина — игра мгновенно завершается поражением. Если клетка безопасна, то бражается количество мин в соседних клетках. Если вокруг ячейки мин нет, запускается аппаратный/программный алгоритм рекурсивного открытия пустых областей.

2.4. Действие «Поставить флаг»

Игрок может пометить закрытую ячейку флажком. При этом уменьшается счетчик доступных флажков. Повторное нажатие снимает флаг с ячейки. На ячейку с флагом действует защита от случайного открытия.

2.5. Условия завершения игры:

Победа: наступает в момент, когда все 10 мин успешно помечены флажками (счетчик мин mines равен 0).

Поражение: наступает при попытке открыть ячейку с миной. При проигрыше все мины на поле принудительно подсвечиваются.

3. Спецификация аппаратных модулей (Hardware Architecture)

Архитектура системы состоит из процессорного блока CdM-8 и набора специализированных подсистем:

3.1. Модуль Центрального Контроллера (processor_handler)

Главное вычислительное ядро схемы.

3.1.1. Функции

Содержит ОЗУ состояния игрового поля, регистры положения курсора, счетчики флагов и мин. Координирует работу памяти команд и памяти данных (Гарвардская архитектура), управляет загрузкой уровней.

3.1.2. Интерфейс

Выдает на видеоконтроллер 32-битный сигнал row_out для отрисовки текущей строки.

3.2. Селектор Карт (map_selector)

3.2.1. Компоненты

Содержит 4 встроенных ПЗУ (ROM) объемом 256 байт каждое. Хранит шаблоны карт уровней (до 8 предустановленных конфигураций сложности).

3.2.2. Логика

Принимает адрес ячейки addr, умножает его на константу 0x40 с помощью умножителя, суммирует со значением выбора уровня sel и возвращает байт состояния cell_info. Дополнительно выдает сигнал seed для потенциального генератора случайных чисел.

3.3. Контроллер Ввода Игрока (controller)

3.3.1. Входы

Физические кнопки up, down, left, right, open (\$O\$), flag (\$F\$), системные линии reset, start, win, lose и тактовый сигнал clk.

3.3.2. Внутренняя логика

Включает аппаратные приоритетные арбитры для исключения дребезга и одновременного нажатия нескольких клавиш, а также D-триггеры для фиксации событий.

3.3.3. Выходы

Генерирует прерывание на процессор через линию IRQ и выдает 3-битный вектор команды `vec`, расшифровывающий конкретное действие игрока.

3.4. Видеоподсистема (`video_handler` + `row_drawer` + `cell_drawer`)

3.4.1. `cell_drawer`

Низкоуровневый отрисовщик одной клетки (размер 7*8 пикселей). Содержит ПЗУ знаков (графические образы закрытой клетки, пустого поля, цифр от 1 до 8, мины и флага). Подсвечивает клетку при наличии активного сигнала `selected`.

3.4.2. `row_drawer`

Объединяет 8 модулей `cell_drawer` в одну горизонтальную линию. Принимает 32-битное слово строки, разбивает его и формирует итоговую шину на 64 пикселя в ширину. Принимает управляющий сигнал `force`, принудительно открывающий все мины.

3.4.3. `video_handler`

Синхронизирует развертку. На основе 3-битного кода строки через дешифраторы генерирует независимые тактовые частоты `clk0...clk7`, поочередно активируя строки на матричном дисплее во избежание мерцания.

3.5. Информационное табло (`info_display` + `symb_drawer`)

Текстовый блок на базе ПЗУ символов для вывода текущего количества оставшихся флажков. В случае победы аппаратно переключает дисплей на статичный вывод слова WIN, в случае поражения — LOSE.

3.6. Автозагрузчик карты (`map_loader`)

Аппаратный автомат (счетчик до 64 и компаратор), который при системном сбросе (`reset`) блокирует процессор, за 64 такта частоты `clk` копирует данные из выбранного ПЗУ карт в ОЗУ игрового поля и затем поднимает флаг готовности `loaded`.

4. Архитектура программного обеспечения (Software Design)

Программа пишется на ассемблере CdM-8 (поддерживающем прерывания) и жестко структурирована в памяти:

4.1. Распределение памяти (Memory Map)

asect 0x00: Стартовая точка программы. Содержит безусловный переход к главной процедуре main.

Переменные (Data RAM):

cursor (1 байт) — текущий индекс выбранной клетки (\$0 \dots 63\$).

mines (1 байт) — счетчик неразминированных мин.

flags (1 байт) — количество оставшихся у игрока флажков.

map — массив (сжатая карта поля), где каждые два бита кодируют одну ячейку (в 1 байте упакована информация о 4 клетках).

asect 0xf0: Размещение Таблицы векторов прерываний. Векторы должны указывать на подпрограммы-обработчики кнопок управления (left, right, up, down, open, flag, start).

4.2. Ключевые алгоритмы и подпрограммы

main и main_loop: инициализирует указатель стека (setsp), вызывает сброс параметров (reset), включает прерывания и уходит в бесконечный цикл ожидания прерываний (команда останова до прихода IRQ).

reset / start: записывает начальное число 10 в ячейки mines и flags, обнуляет координату cursor.

left / right / up / down: Обработчики движения. Извлекают cursor, применяют битовую маску, проверяют границы и инкрементируют/декрементируют адрес. Возврат осуществляется строго через rti.

Битовое декодирование (set_cells_data и get_cell_masks):

Поскольку данные упакованы по 4 ячейки на байт, set_cells_data делит текущий индекс курсора на 4, вычисляет смещение относительно базы map и извлекает нужный байт карты.

`get_cell_masks` берет остаток от деления, определяет точную позицию внутри байта и через цикл `while` со сдвигами подготавливает точные маски для флага и для мины.

Обработка клика (`open`): извлекает маски. Проверяет наличие флага (если установлен — игнорирует клик и выходит). Проверяет наличие мины (если есть — выполняет переход на секцию `lose`). Если клетка безопасна — запускает логику подсчета окружения и отрисовки.

Установка флага: проверяет доступность флага в счетчике. С помощью логической операции XOR инвертирует бит флага в байте памяти `map`. Обновляет счетчики `mines` и `flags`. Если `mines` достиг нуля — осуществляет переход на процедуру `win`.

5. Критерии приемки проекта

Схема в Logisim успешно запускается, процесс аппаратной автозагрузки уровня (`map_loader`) корректно заполняет начальные массивы.

Кнопки навигации безошибочно перемещают курсор по сетке дисплея 8*8 во всех направлениях.

Игра корректно реагирует на подрыв на мине (вывод `LOSE` на табло + открытие всех скрытых мин) и на успешное разминирование поля (вывод `WIN`).

В коде ассемблера отсутствуют пересечения области стека и области исполняемых инструкций. По окончании обработки каждого прерывания управление возвращается корректно с восстановлением регистров из стека.